



INSTITUT INFORMATIQUE SUD AVEYRON

Reclassement professionnel

Développement web back-end

Découverte des variables en PHP

Les opérateurs et la concaténation en PHP

Sommaire

1	Introduction aux variables	2
1.1	Qu'est-ce qu'une variable ?	2
1.2	Les règles de déclaration des variables	2
1.3	Les opérateurs d'affectation et de comparaison	2
2	Les types de données de PHP	3
2.1	Le type chaîne de caractères (string)	5
2.2	Les types entier (integer) et décimal (double ou float)	5
2.3	Le type boolean	6
2.4	Le type null	6
2.5	Les types de données tableau (array) et objet (object)	7
2.6	Le type de données ressource (resource)	7
3	Les opérateurs et la concaténation	7
3.1	Les opérateurs de chaînes et la concaténation en PHP	7
3.2	Les opérateurs arithmétiques	9
3.3	Les opérateurs d'affectation et opérateurs combinés	10

1 Introduction aux variables

1.1 Qu'est-ce qu'une variable ?

Une variable est un conteneur servant à stocker une information de manière temporaire, comme une chaîne de caractère, un nombre ...

Le propre d'une variable est de pouvoir varier, c'est-à-dire que son contenu doit pouvoir évoluer au fil de l'exécution d'un programme.

En PHP, les variables ne servent à stocker une information que temporairement ? C'est-à-dire que la variable ne va exister que durant le temps d'exécution du script l'utilisant.

1.2 Les règles de déclaration des variables

Lorsqu'on crée une variable, on dit également que l'on « déclare » une variable.

On va pouvoir choisir le nom qu'on souhaite pour nos variables. Cependant, il y a quelques règles à respecter et à connaître lors de la déclaration d'une variable :

- Toute variable en PHP doit commencer par le signe `$` et sera suivi du nom de la variable.
- Le nom d'une variable doit commencer par une lettre ou un underscore (`_`) et ne doit pas commencer par un chiffre.
- Le nom d'une variable ne doit contenir que des lettres, des chiffres et des underscores mais pas de caractères spéciaux.
- Le nom d'une variable ne doit pas contenir d'espace.

Notez qu'en PHP, le nom des variables est sensible à la casse. `$texte`, `$Texte` et `$TEXTE` sont des variables différentes.

Enfin, sachez qu'il existe des noms « réservés » en PHP. Vous ne pouvez pas utiliser ces noms comme noms pour vos variables, tout simplement car le langage PHP les utilise déjà pour désigner différents objets intégrés au langage. <https://www.php.net/manual/fr/reserved.php>

1.3 Les opérateurs d'affectation et de comparaison

L'affectation d'une valeur à une variable se fait à l'aide du signe `=`.

L'égalité en termes de valeurs simples est symbolisée en PHP par un double signe égal `==`.

L'égalité en termes de valeurs et de types de données est symbolisée par un triple signe égal `===`.

Examinons le code ci-dessous :

```
</head>
<body>
  <h1>Les opérateurs d'affectation et de comparaison</h1>
  <?php
    $a = 12;
    $b = 12;
    $c = "12";
    $d = 10;
```

```
var_dump($a == $b); // retourne vrai
echo "<b>";
var_dump($a == $c); // retourne vrai :-(
echo "<b>";
var_dump($a === $c); // retourne faux
echo "<b>";
var_dump($a == $d); // retourne faux

?>
</body>
</html>
```

Voici ce qu'il nous retourne.



Attention donc à bien prendre en compte cette permissivité de PHP.

2 Les types de données de PHP

En PHP, contrairement à d'autres langages de programmation, il n'est pas nécessaire de préciser à priori le type d'une variable. PHP va détecter le type de la valeur à stocker.

Le type de la variable va donc pouvoir changer en fonction de la valeur qui lui est affectée.

Ce comportement est à prendre en considération car au fil de l'exécution, une variable peut changer de type et par le fait amener de la confusion.

Les variables en PHP vont pouvoir stocker 8 grands types de données différents :

- Le type `string` ou « chaîne de caractère »
- Le type `integer` ou « nombre entier »
- Le type `float` ou « nombre décimal »
- Le type `boolean` ou « booléen »
- Le type `array` ou « tableau »

- Le type `null`
- Le type `objet`
- Le type `resource`

Nous n'aborderons que les types « simples » de valeurs.

Examinons le code suivant :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cours PHP</title>
</head>
<body>
  <h1>Les types de variables en PHP</h1>
  <?php
    $a;
    echo 'Le type de $a est : ' . gettype($a);
    echo "<br>";
    $a = 12;
    echo 'Le type de $a est : ' . gettype($a);
    echo "<br>";
    $a = 12.5;
    echo 'Le type de $a est : ' . gettype($a);
    echo "<br>";
    $a = "toto";
    echo 'Le type de $a est : ' . gettype($a);
    echo "<br>";
    $a = false;
    echo 'Le type de $a est : ' . gettype($a);
    echo "<br>";
    $a = ["toto", 1, true];
    echo 'Le type de $a est : ' . gettype($a);
    echo "<br>";
  ?>
</body>
</html>
```

L'instruction `gettype()` retourne le type d'une variable.

Qui nous retourne ceci à l'exécution :



2.1 Le type chaîne de caractères (string)

Le type **string** ou « chaîne de caractères » est une séquence de caractères ou plus simplement du texte.

Notez que toute valeur stockée en utilisant des guillemets " ou des apostrophes ' sera considéré comme une **string** et ceci même si ce sont des nombres.

Voici du code pour nous en convaincre :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cours PHP</title>
</head>
<body>
  <h1>Les types de variables en PHP</h1>
  <?php
    $texte = "Mon nom et toto"; // stocke du texte
    echo 'Le type de $texte est: ' . gettype($texte);
    echo '<br>';
    $age = 28; // stocke un nombre
    echo 'Le type de $age est: ' . gettype($age);
    echo '<br>';
    $age2 = '28'; // stocke du texte
    echo 'Le type de $age2 est: ' . gettype($age2);
    echo '<br>';
  ?>
</body>
</html>
```

Et le résultat à l'exécution :



2.2 Les types entier (integer) et décimal (double ou float)

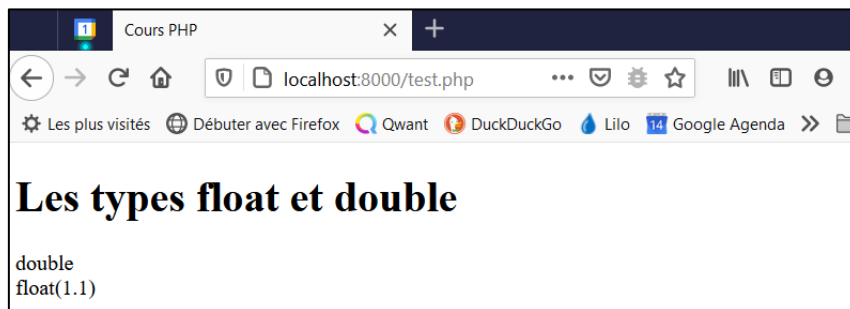
En PHP, on va pouvoir stocker deux types différents de données numériques dans nos variables :

- Le type **integer**, qui concerne tous les entiers positifs ou négatifs.
- Le type **double** ou **float**, qui concerne tous les nombres décimaux positifs ou négatifs. Attention, il faut utiliser la notation anglo-saxonne et utiliser le point . et non la virgule ,.

En PHP, il n'y a pas de différence entre **float** et **double**. [Explication](#).

Pour nous en convaincre :

```
<?php
    $a = 1.1;
    echo gettype($a);
    echo '<br>';
    var_dump($a);
?>
```



2.3 Le type boolean

Le type **boolean** est un type qui ne peut contenir que deux valeurs **true** (vrai) ou **false** (faux). Ce type n'est pas courant dans la vie de tous les jours mais est largement utilisé en informatique. N'oublions pas que l'informatique n'est composée que de **1** et de **0** donc de **true** et de **false**.

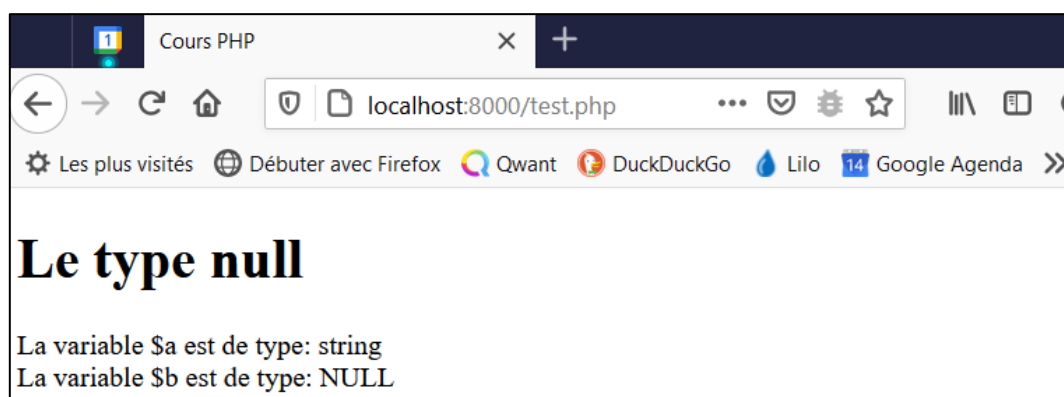
2.4 Le type null

Le type **null** est un type de données assez particulier puisqu'il représente une absence de valeur et sert donc à représenter des variables vides en PHP.

Examinons ce code :

```
<?php
    $a = ''; // chaine vide mais présente
    $b; // absence de valeur
    echo 'La variable $a est de type: ' . gettype($a);
    echo '<br>';
    echo 'La variable $b est de type: ' . gettype($b);
?>
```

Le retour n'est donc pas une surprise :



2.5 Les types de données tableau (array) et objet (object)

Les type de données `array` et `object` sont des types de données complexes et seront étudiés dans des chapitres dédiés.

2.6 Le type de données ressource (resource)

Une ressource est une variable particulière qui contient une référence vers une ressource externe au PHP, comme dans le cas d'une variable qui représente la connexion vers une base de données par exemple.

3 Les opérateurs et la concaténation

Un opérateur est un symbole qui va être utilisé pour effectuer certaines actions en particulier sur des variables et des valeurs.

Par exemple, l'opérateur `+` va permettre l'addition de deux nombres alors que l'opérateur `=` va permettre l'affectation d'une valeur à une variable.

Il existe bon nombre [d'opérateurs en PHP](#) mais nous nous concentrerons sur les opérateurs :

- Arithmétiques
- De chaînes
- D'affectation

Nous profiterons de l'occasion pour préciser les différences entre l'utilisation des apostrophes `'` et des guillemets `"` lorsqu'on manipule une valeur de type `string`.

3.1 Les opérateurs de chaînes et la concaténation en PHP

En programmation, concaténer signifie « mettre bout à bout ». En PHP, l'opérateur de concaténation est le point `.`.

Cet opérateur va s'avérer particulièrement utile lorsqu'on voudra stocker le contenu de plusieurs variables dans une seule variable ou pour afficher plusieurs contenus au sein d'une seule instruction `echo`.

Pour bien comprendre comment fonctionne l'opérateur de concaténation et son intérêt, il me semble nécessaire de connaître les différences entre l'utilisation des guillemets et des apostrophes lorsqu'on manipule une chaîne de caractères en PHP.

Sur ce sujet, vous pouvez retenir que la différence majeure entre l'utilisation des guillemets `"` et d'apostrophes `'` est que tout ce qui est entre guillemets `"` va être interprété (remplacé par sa valeur) tandis que quasiment tout ce qui est entre apostrophes `'` va être considéré comme une chaîne de caractères.

En PHP, le caractère d'échappement est le `\`. Il sert à dire à PHP de ne pas interpréter le caractère qui le suit.

Examinons le code ci-dessous ainsi que le résultat affiché :

```
<!DOCTYPE html>
<html lang="en">
```



```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cours PHP</title>
</head>
<body>
  <h1>Les guillemets et apostrophes</h1>
  <?php
    $prenom = "John";
    $nom = "Doe";
    $age = 35;

    echo "Je m'appelle $prenom et j'ai $age ans <br>";
    echo "Je m'appelle {$prenom} et j'ai {$age} ans <br>";
    echo 'Je m\'appelle $prenom et j\'ai $age ans <br>';

    $sortie1 = "Je suis $prenom $nom, j'ai $age ans <br>";
    $sortie2 = "Je suis {$prenom} {$nom}, j'ai {$age} ans <br>";
    $sortie3 = 'Je suis $prenom $nom, j\'ai $age ans';

    echo $sortie1;
    echo $sortie2;
    echo $sortie3;

  ?>
</body>
</html>
```



Ici nous avons trois variables `$prenom`, `$nom` et `$age`.

Quels enseignements pouvons-nous tirer de cet exemple ?

- Utiliser `echo` directement ou construire une variable et faire un `echo` de cette variable produit le même résultat.
- Toutes les variables présentes entre guillemets " sont interprétées.
- Pour plus de visibilité, on préférera la version avec la variable entre accolades `{ }`.
- Toutes les variables présentes entre apostrophes ' ne sont pas interprétées.
- Pour échapper un caractère on utilise `\`.

Pour la concaténation, examinons le code et la sortie ci-dessous :

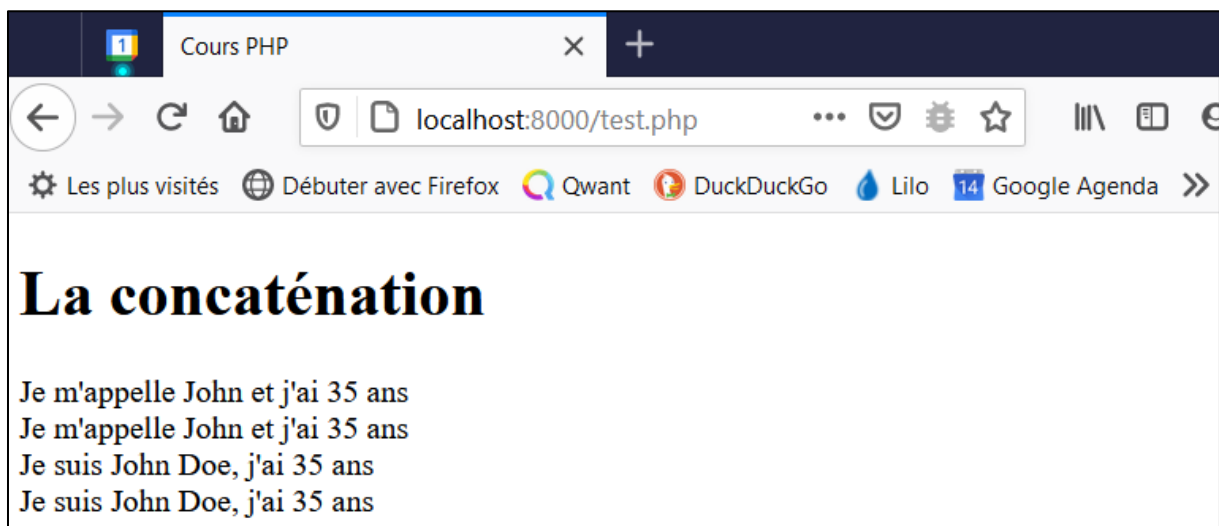
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cours PHP</title>
</head>
<body>
  <h1>La concaténation</h1>
  <?php
    $prenom = "John";
    $nom = "Doe";
    $age = 35;

    echo "Je m'appelle " . $prenom . " et j'ai " . $age . " ans <br>";
    echo 'Je m\'appelle ' . $prenom . ' et j\'ai ' . $age . ' ans
  <br>';

    $sortie1 = "Je suis " . $prenom . " " . $nom . ", j'ai " . $age . "
ans <br>";
    $sortie2 = 'Je suis ' . $prenom . ' ' . $nom . ', j\'ai ' . $age .
' ans';

    echo $sortie1;
    echo $sortie2;

  ?>
</body>
</html>
```



Pas de réelle surprise mais c'est moins lisible. Nous sommes obligé de construire complètement notre sortie.

3.2 Les opérateurs arithmétiques

Les opérateurs arithmétiques vont nous permettre d'effectuer des opérations mathématiques sur les valeurs numériques des variables.

Opérateur	Nom de l'opération associée
-----------	-----------------------------

+	Addition
-	Soustraction
*	Multiplication
/	Division
%	Modulo (reste d'une division euclidienne)
**	Exponentielle (élévation à la puissance d'un nombre par un autre)

L'ordre de priorité est le même qu'en mathématique :

- Parenthèses
- Exponentielle
- Multiplication, division, modulo
- Addition, soustraction

A noter que l'évaluation des opérations se fait de gauche à droite sauf pour l'exponentielle qui est évalué de droite à gauche. Si on écrit `$x = 3 ** 2 ** 3`, la variable `$x` stockera **6561** qui correspond à `3 ** 8` puisqu'on commence par calculer `2 ** 3 = 8`.

3.3 Les opérateurs d'affectation et opérateurs combinés

Comme indiqué précédemment l'opérateur d'affectation en PHP est le signe `=`. Cependant, il existe aussi des opérateurs combinés qui vont permettre d'effectuer une opération mathématique ou une concaténation et ensuite d'affecter le résultat à une variable.

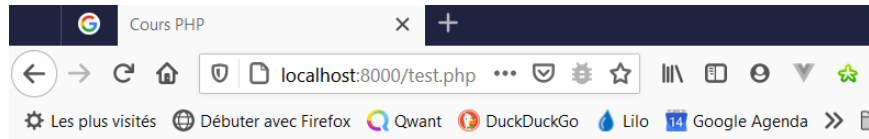
Opérateur	Définition
<code>.=</code>	Concatène puis affecte le résultat
<code>+=</code>	Additionne puis affecte le résultat
<code>-=</code>	Soustrait puis affecte le résultat
<code>*=</code>	Multiplie puis affecte le résultat
<code>/=</code>	Divise puis affecte le résultat
<code>%=</code>	Calcule le modulo puis affecte le résultat
<code>**=</code>	Elève à la puissance puis affecte le résultat

Examinons un exemple de code et son résultat dans le navigateur :

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cours PHP</title>
</head>
<body>
  <h1>Les opérateurs combinés</h1>
  <?php
    $a = "Bonjour";
    $a .= " le monde"; // $a stocke "Bonjour le monde"
    echo '$a stocke: ' . $a . '<br>';

    $x = 5;
    $x -= 3; // $x stocke 2
    echo '$x stocke: ' . $x . '<br>';
```

```
$y = 3;  
$y **= $x; // $y stocke 9  
echo '$y stocke: ' . $y . '<br>';  
?  
</body>  
</html>
```



Les opérateurs combinés

\$a stocke: Bonjour le monde
\$x stocke: 2
\$y stocke: 9